Technical University of Cluj-Napoca, Romania
Department of Computer Science

# Connecting to a database in Java
# Java Refelction

# **J**ava **D**ata**b**ase **C**onnectivity (JDBC)

- Is a java API to connect and execute query with the database
- Uses jdbc drivers to connect with the database
- Includes APIs for:
  - Making a connection to a database
  - Creating SQL or MySQL statements
  - Executing SQL or MySQL queries in the database
  - Viewing & Modifying the resulting records

# Steps to connect a java application with the database

- Register the driver
- Establish the connection
- Creating statement
- Executing queries
- Closing connection

# Steps to connect a java application with the database
# Register the driver

- Use the **forName**() method of **Class** class

  **public static void forName**(**String** className)**throws**

  **ClassNotFoundException**

  - This method is used to dynamically load the driver
- Example to register the driver class for MySQL database

  **Class**.**forName**(“com.mysql.jdbc.Driver");

# Steps to connect a java application with the database Create the connection object

- To establish connection with the database use the **getConnection**() method of **DriverManager** class

**public static Conne**ction getConnection(**String** url) **throws SQLException**

**public static Connection** getConnection(**String** url, **String** name, **String** password) **throws SQLException**

- Example to establish connection with a mysql database

**Connection** con = **DriverManager**.getConnection(

“jdbc:mysql://localhost/arhiva",“user","password");

# Steps to connect a java application with the database
# Create the statement object

- To create statement use the **createStatement**() method of **Connection** interface

**public Statement** createStatement() **throws SQLException**

  - The object of statement is responsible to execute queries with the database

- Example to create the statement object

**Statement** *stmt* = *con*.c**reateStatement**();

# Steps to connect a java application with the database Execute the query

- To execute queries to the database use the **executeQuery**() method of **Statement** interface

  **public ResultSet executeQuery**(**String** sql)**throws SQLException**

  - This method returns the object of **ResultSet** that can be used to get all the records of a table

- Example to execute query

```
ResultSet rs = stmt.executeQuery("select * from emp");
while(rs.next()){
  System.out.println(rs.getInt(1)+" "+rs.getString(2)); }
```

# Steps to connect a java application with the database Close the connection object

- By closing connection, object statement and ResultSet will be closed automatically
- Use the **close**() method of **Connection** interface to close the connection

**public void close**()**throws SQLException**

- Example to close connection

*con*.**close**();

# Example of connecting to a mysql database (I)

- Necessary information that need to know for connection:
  - Driver class for the mysql database

    **com.mysql.jdb.Driver**
  - Connection URL for the mysql database

    **jdbc:mysql://localhost:3306/sonoo**
    - where : (i) **jdbc** is the API; (ii) **mysql** is the database; (iii) **localhost** is the server name on which mysql is running, we may also use IP address; (iv) **3306** is the port number ; (v) **sonoo** is the database name
  - Username
    - The default username for the mysql database is **root**
  - Password
    - **Password** given by the user for the mysql database

# Example to connect to a mysql database (II)

- Example of creating a database with a table

```
create database sonoo;
use sonoo;
create table emp(id int(10),name varchar(40),age int(3));
```

# Example to connect to a mysql database (III)

```
import java.sql.*;
class MysqlCon{
public static void main(String args[]){
try{
Class.forName("com.mysql.jdbc.Driver");
Connection con=DriverManager.getConnection(
"jdbc:mysql://localhost:3306/sonoo","root","root");
//here sonoo is database name, root is username and password
Statement stmt=con.createStatement();
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next())
System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getString(3));
con.close();
}catch(Exception e){ System.out.println(e);}
}
}
```

# mysqlconnector.jar

- To connect java application with a **mysql** databas, **mysqlconnector.jar** file is required to be loaded
  - It can be added either as an external jar file dependency
  - It can be added as maven dependency, in case of a Maven project

```
<dependencies>

    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>6.0.6</version>
    </dependency>


</dependencies>
```

# mysqlconnector.jar

- The Java application uses this external library to communicate with the MySQL server
- It sends queries to the server using **Statements** and it receives the results of the queries as **ResultSet**

# DriverManager class

- Acts as an interface between user and drivers
- Establish a connection between a database and the appropriate driver

| Method | Description |
|---|---|
| 1) public static void registerDriver(Driver driver): | is used to register the given driver with DriverManager. |
| 2) public static void deregisterDriver(Driver driver): | is used to deregister the given driver (drop the driver from the list) with DriverManager. |
| 3) public static Connection getConnection(String url): | is used to establish the connection with the specified url. |
| 4) public static Connection getConnection(String url,String userName,String password): | is used to establish the connection with the specified url, username and password. |

# Connection interface

- A **Connection** is the session between java application and database
- Object of **Connection** can be used to get the object of **Statement**
- Commonly used method from **Connection** interface

**1) public Statement createStatement():** creates a statement object that can be used to execute SQL queries.

# Statement interface

- The **Statement** interface provides methods to execute queries with the database
- It provides a method to get the object of **ResultSet**
- Commonly used method from **Statement** interface

**1) public ResultSet executeQuery(String sql):** is used to execute SELECT query. It returns the object ResultSet.

**2) public int executeUpdate(String sql):** is used to execute specified query, it may be create, dro insert, update, delete etc.

# Statement interface Example

```
import java.sql.*;
class FetchRecord{
public static void main(String args[])throws Exception{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
Statement stmt=con.createStatement();

//stmt.executeUpdate("insert into emp765 values(33,'Irfan',50000)");
//int result=stmt.executeUpdate("update emp765 set name='Vimal',salary=10000 where id=33");
int result=stmt.executeUpdate("delete from emp765 where id=33");
System.out.println(result+" records affected");
con.close();
}}
```

# ResultSet Interface

- A **ResultSet** object maintains a cursor pointing to its current row of data
- Initially the cursor is positioned before the first row
- The **next** method moves the cursor to the next row
  - It returns false when there are no more rows in the **ResultSet** object
  - It can be used in a while loop to iterate through the result set

# ResultSet Interface
# Navigating a ResultSet

**public boolean previous() throws SQLException**

Moves the cursor to the previous row. This method returns false if the previous row is off the result set.

**public boolean next() throws SQLException**

Moves the cursor to the next row. This method returns false if there are no more rows in the result set.

**public boolean first() throws SQLException**

Moves the cursor to the first row.

**public void last() throws SQLException**

Moves the cursor to the last row.

# ResultSet Interface
# Viewing a Result Set

- **ResultSet** interface contains methods for getting the data of the current row
- There is a get method for each of the possible data types, and each get method has two versions
  - One that takes in a column name
  - One that takes in a column index

# ResultSet Interface
# Viewing a Result Set

**public int getInt(String columnName) throws SQLException**

Returns the int in the current row in the column named columnName.

**public int getInt(int columnIndex) throws SQLException**

Returns the int in the current row in the specified column index. The column index starts at 1, meaning the first column of a row is 1, the second column of a row is 2, and so on.

# ResultSet Interface
# Viewing a Result Set

- In the **ResultSet** interface there are **get** methods for
  - Each of the eight Java primitive types
  - Common types such as:
    - java.lang.String
    - java.lang.Object
    - java.net.URL

# ResultSet Interface
# Updating a Result Set

- **ResultSet** interface contains a collection of update methods for updating the data of a result set
- There are two update methods for each data type
  - One that takes in a column name
  - One that takes in a column index

# ResultSet Interface
# Updating a Result Set

**public void updateString(int columnIndex, String s)**
**throws SQLException**

Changes the String in the specified column to the value of s.

**public void updateString(String columnName, String s)**
**throws SQLException**

Similar to the previous method, except that the column is specified by its name instead of its index.

- There are update methods for
  - the eight primitive data types
  - String, Object, URL, and the SQL data types in the java.sql package

# ResultSet Interface
# Updating a Result Set

- Methods for updating/deleting a row in the result set

**public void updateRow()**

Updates the current row by updating the corresponding row in the database.

**public void deleteRow()**

Deletes the current row from the database

# ResultSet Interface Example

```
import java.sql.*;
class FetchRecord{
public static void main(String args[])throws Exception{

Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
Statement stmt=con.createStatement( );
ResultSet rs=stmt.executeQuery("select * from emp765");

//getting the record of 3rd row
rs.absolute(3);
System.out.println(rs.getString(1)+" "+rs.getString(2)+" "+rs.getString(3));

con.close();
}}
```

# PreparedStatement interface

- Is a subinterface of **Statement**
- It is used to execute parameterized query
- Example of parameterized query:

```
String sql="insert into emp values(?,?,?)";
```

  - We are passing parameter (?) for the values
  - Its value will be set by calling the setter methods of **PreparedStatement**

# PreparedStatement interface

- How to get the instance of PreparedStatement?
  - Use the **prepareStatement**() method of **Connection** interface to return the object of **PreparedStatement**

```
public PreparedStatement prepareStatement(String query)throws
SQLException{}
```

# PreparedStatement interface

| Method | Description |
|---|---|
| public void setInt(int paramIndex, int value) | sets the integer value to the given parameter index. |
| public void setString(int paramIndex, String value) | sets the String value to the given parameter index. |
| public void setFloat(int paramIndex, float value) | sets the float value to the given parameter index. |
| public void setDouble(int paramIndex, double value) | sets the double value to the given parameter index. |
| public int executeUpdate() | executes the query. It is used for create, drop, insert update, delete etc. |
| public ResultSet executeQuery() | executes the select query. It returns an instance o ResultSet. |

# PreparedStatement interface Example of inserting a record

- We consider the following table:

create table emp(id number(10),name varchar2(50));

```
import java.sql.*;
class InsertPrepared{
public static void main(String args[]){
try{
Class.forName("oracle.jdbc.driver.OracleDriver");

Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","oracle");

PreparedStatement stmt=con.prepareStatement("insert into Emp values(?,?)");
stmt.setInt(1,101);//1 specifies the first parameter in the query
stmt.setString(2,"Ratan");

int i=stmt.executeUpdate();
System.out.println(i+" records inserted");

con.close();

}catch(Exception e){ System.out.println(e);}

}
}
```

# PreparedStatement interface Example of deleting a record

```
PreparedStatement stmt=con.prepareStatement("delete from emp where id=?");

stmt.setInt(1,101);

int i=stmt.executeUpdate();

System.out.println(i+" records deleted");
```

# PreparedStatement interface
# Example of retrieving the records of a table

```
PreparedStatement stmt=con.prepareStatement("select * from emp");

ResultSet rs=stmt.executeQuery();

while(rs.next()){

System.out.println(rs.getInt(1)+" "+rs.getString(2));

}
```

# Java Reflection

- It possible to inspect classes, interfaces, fields and methods at runtime, without knowing the names of the classes, methods etc. at compile time
- It is also possible to instantiate new objects, invoke methods and get/set field values
- From the classes you can obtain information about
  - Class Name, Class Modifies (public, private, synchronized etc.), Package Info, Superclass, Implemented Interfaces, Constructors, Methods, Fields, Annotations

# Java Reflection -The Class Object

- Before you can do any inspection on a class you need to obtain its **java.lang.Class** object by
  - Using the method **getClass()** of the class **Object (**if you have a reference to an object)

    **Class** myObjectClass = **MyObject**.**class**

  - Using the static method **forName()** of the class **Class** (when the type is available, but no such object is available)

    **Class.forName**(*className*);

# Java Reflection -The Class Object

- **Class Name**
  - Can be obtained from a **Class** object using the **getName**() method

```
Class aClass = ... //obtain Class object.
String className = aClass.getName();
```

```
Horse minzu = new Horse();
Class cHorse = minzu.getClass();
String className = cHorse.getName();
```

# Java Reflection -The Class Object

- Modifiers of a class
  - Can be obtained from a **Class** object using the **getModifiers**() method

```
Class  aClass = ... //obtain Class object. See prev. section
int modifiers = aClass.getModifiers();
```

  - getModifiers()
    - Returns the Java language modifiers for this class or interface, encoded in an integer
    - The modifiers consist of the JVM's constants for public, protected, private, final, static, abstract and interface; they should be decoded using the methods of class **Modifier**.

# Java Reflection -The Class Object

- Superclass of a class
  - Can be obtained from a **Class** object using the **getSuperClass**() method

    **Class** superclass = aClass.**getSuperclass**();
- Constructors of a class
  - You can access the constructors of a class with **getConstructors**() method:

    **Constructor**[] constructors = aClass.**getConstructors**();
- Methods of a class
  - You can access the methods of a class like this:

    **Method**[] method = aClass.**getMethods**();

# Java Reflection -The Class Object

- Fields
  - You can access the fields (member variables) of a class like this:
    - Field[] method = aClass.getFields();

# Java Reflection - Obtaining Constructor Objects

```
Class aClass = ...//obtain class object
Constructor[] constructors = aClass.getConstructors();
```

- The Constructor[] array will have one **Constructor** instance for each public constructor declared in the class
- If you know the precise parameter types of a constructor you can obtain this constructor as follow:

```
Class aClass = ...//obtain class object
Constructor constructor =aClass.getConstructor(new Class[]{String.class});
```

  - This example returns the public constructor of the given class which takes a **String** as parameter

# Java Reflection - Obtaining Constructor Objects

- Constructor Parameters
  - You can read what parameters a given constructor takes like this:

```
Constructor constructor = ... // obtain constructor
Class[] parameterTypes = constructor.getParameterTypes();
```

# Java Reflection - Obtaining Constructor Objects

- Instantiating Objects using **Constructor** Object
  - You can instantiate an object like this:

```
//get constructor that takes a String as argument
Constructor constructor = MyObject.class.getConstructor(String.class);
MyObject myObject = (MyObject) constructor.newInstance("constructor-arg1");
```

  - The **Constructor.newInstance**() method
    - Takes an optional amount of parameters, but you must supply exactly one parameter per argument in the constructor you are invoking
    - In this case it was a constructor taking a String, so one String must be supplied

# Java Reflection - Obtaining Field Objects

- The **Field** class is obtained from the **Class** object as follows:

```
Class aClass = ...//obtain class object
Field[] fields = aClass.getFields();
```

  - The **Field**[] array will have one **Field** instance for each public field declared in the class

# Java Reflection - Obtaining Field Objects

- If you know the name of the field you want to access, you can access it like this:

```
MyObject ob= MyObject();
Class  aClass = ob.getClass();
Field field = aClass.getField("someField");
```

- This example return the **Field** instance corresponding to the field **someField** as declared in the **MyObject** class:

```
public class MyObject{
  public String someField = null;
 }
```

# Java Reflection - Obtaining Field Objects

- Field Name
  - The name of a **Field** is obtained with **getName**() method from **Field** class

    **Field** field = ... //obtain field object

    String fieldName = **field.getName**();
- Field Type
  - The field type (String, int etc.) of a field is determined using the **getType**() method from Field class:

    **Field** field = aClass.**getField**("someField");

    Object fieldType = field.**getType**();

# Java Reflection -Obtaining Field Objects

- **Getting and Setting Field Values**
  - You can get and set The field value using the get() and set()methods from **Field** class

```
MyObject ob= MyObject();
Class  aClass = ob.getClass();
Field field = aClass.getField("someField");
MyObject objectInstance = new MyObject();
Object value = field.get(objectInstance);
field.set(objetInstance, value);
```

  - The objectInstance parameter passed to the get and set method should be an instance of the class that owns the field
  - In this example an instance of MyObject is used, because the **someField** is an instance member of the MyObject class

# Java Reflection -Obtaining Field Objects

- **Getting and Setting Field Values**
  - It the field is a static field (public static ...) pass null as parameter to the get and set methods, instead of the **objectInstance** parameter

# Java Reflection - Obtaining Method Objects

- The Method class is obtained from the **Class** object with getMtehods();

```
Class aClass = ...//obtain class object
Method[] methods = aClass.getMethods();
```

  - The **Method**[] array will have one **Method** instance for each public met**h**od declared in the class

# Java Reflection - Obtaining Method Objects

- If you know the precise parameter types of the method you want to access, you can do as in example bellow:

```
Class  aClass = ...//obtain class object
Method method = aClass.getMethod("doSomething", new
Class[]{String.class});
```

  - This example returns the public method named "doSomething", in the given class which takes a String as parameter

# Java Reflection - Obtaining Method Objects

- If the method you are trying to access takes no parameters, pass null as the parameter type array, like this:

```
Class  aClass = ...//obtain class object
Method method = aClass.getMethod("doSomething", null);
```

# Java Reflection - Obtaining Method Objects

- Method Parameters and Return Types
  - You can read what parameters a given method takes like this:

```
Method method = ... // obtain method - see above
Class[] parameterTypes = method.getParameterTypes();
```

- You can access the return type of a method like this:

```
Method method = ... // obtain method - see above
Class returnType = method.getReturnType();
```

# Java Reflection - Invoking Methods using Method Object

- You can invoke a method like this:

```
//get method that takes a String as argument
Method method = MyObject.class.getMethod("doSomething", String.class);
Object returnValue = method.invoke(null, "parameter-value1");
```

  - The null parameter is the object you want to invoke the method on.
  - If the method is static you supply null instead of an object instance.
  - In this example, if doSomething(String.class) is not static, you need to supply a valid MyObject instance instead of null;

# Java Reflection - Accessing Private Fields

- To access a private field you will need to call
  - Class.getDeclaredField(String name)
  - Class.getDeclaredFields() method

```
public class PrivateObject {

  private String privateString = null;

  public PrivateObject(String privateString) {
    this.privateString = privateString;
  }
}
```

```
PrivateObject privateObject = new PrivateObject("The Private Value");

Field privateStringField = PrivateObject.class.
            getDeclaredField("privateString");

privateStringField.setAccessible(true);

String fieldValue = (String) privateStringField.get(privateObject);
System.out.println("fieldValue = " + fieldValue);
```

- The example will print out the text "fieldValue = The Private Value", which is the value of the private field privateString of the PrivateObject instance created at the beginning of the code sample

# Java Reflection - Accessing Private Fields

- getDeclaredField() method
  - Returns fields declared in that particular class, not fields declared in any superclasses
- setAcessible(true) method from Field
  - Turn off the access checks for this particular **Field** instance, for reflection only
  - You can access it even if it is private, protected or package scope, even if the caller is not part of those scopes. You still can't access the field using normal code. The compiler won't allow it.

# Java Reflection - Accessing Private Methods

- To access a private method you will need to call
  - Class.getDeclaredMethod(String name, Class[] parameterTypes)
  - Class.getDeclaredMethods() method

```
public class PrivateObject {

  private String privateString = null;

  public PrivateObject(String privateString) {
    this.privateString = privateString;
  }

  private String getPrivateString(){
    return this.privateString;
  }
}
```

```
PrivateObject privateObject = new PrivateObject("The Private Value")

Method privateStringMethod = PrivateObject.class.
        getDeclaredMethod("getPrivateString", null);

privateStringMethod.setAccessible(true);

String returnValue = (String)
        privateStringMethod.invoke(privateObject, null);

System.out.println("returnValue = " + returnValue);
```

  - This example print out the text "returnValue = The Private Value", which is the value returned by the method getPrivateString() when invoked on the PrivateObject instance created at the beginning of the code sample.

# Java Reflection - Annotations

- Is a new feature from Java 5.
- Are a kind of comment or meta data you can insert in your Java code
- Can then be processed at compile time by pre-compiler tools, or at runtime via Java Reflection.

# Java Reflection - Annotations

- Example of class annotation:

```
@MyAnnotation(name="someName",  value = "Hello World")
public class TheClass {}
```

- The class TheClass has the annotation @MyAnnotation written on top. Annotations are defined like interfaces. Here is the MyAnnotation definition:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface MyAnnotation {
   public String name();
   public String value();}
```

# Java Reflection - Annotations

- The @ in front of the interface marks it as an annotation.
- The two directives in the annotation definition specifies how the annotation is to be used
  - @Retention(RetentionPolicy.RUNTIME) means that the annotation can be accessed via reflection at runtime. @Target(ElementType.TYPE) means that the annotation can only be used ontop of types (classes and interfaces typically).
  - You can also specify METHOD or FIELD, or you can leave the target out all together so the annotation can be used for both classes, methods and fields.

# Java Reflection - Annotations

- Class Annotations
  - You can access the annotations of a class, method or field at runtime
    - Here is an example that accesses the class annotations:

```
Class aClass = TheClass.class;
Annotation[] annotations = aClass.getAnnotations();

for(Annotation annotation : annotations){
    if(annotation instanceof MyAnnotation){
        MyAnnotation myAnnotation = (MyAnnotation) annotation;
        System.out.println("name: " + myAnnotation.name());
        System.out.println("value: " + myAnnotation.value());
    }
}
```

# Java Reflection - Annotations

- Class annotations
  - You can also access a specific class annotation like this:

```
Class aClass = TheClass.class;
Annotation annotation = aClass.getAnnotation(MyAnnotation.class);

if(annotation instanceof MyAnnotation){
    MyAnnotation myAnnotation = (MyAnnotation) annotation;
    System.out.println("name: " + myAnnotation.name());
    System.out.println("value: " + myAnnotation.value());
}
```

# Java Reflection - Annotations

- **Method Annotations**
  - Here is an example of a method with annotations:

```
public class TheClass {
  @MyAnnotation(name="someName",  value = "Hello World")
  public void doSomething(){}
}
```

  - You can access method annotations like this:

```
Method method = ... //obtain method object
Annotation[] annotations = method.getDeclaredAnnotations();

for(Annotation annotation : annotations){
    if(annotation instanceof MyAnnotation){
        MyAnnotation myAnnotation = (MyAnnotation) annotation;
        System.out.println("name: " + myAnnotation.name());
        System.out.println("value: " + myAnnotation.value());
    }
}
```

# Java Reflection - Annotations

- **Method Annotations**
  - You can also access a specific method annotation like this:

```
Method method = ... // obtain method object
Annotation annotation = method.getAnnotation(MyAnnotation.class);

if(annotation instanceof MyAnnotation){
    MyAnnotation myAnnotation = (MyAnnotation) annotation;
    System.out.println("name: " + myAnnotation.name());
    System.out.println("value: " + myAnnotation.value());
}
```

# Java Reflection - Annotations

- **Parameter Annotations**
  - It is possible to add annotations to method parameter declarations too.
  - Here is how that looks:

```
public class TheClass {
  public static void doSomethingElse(
        @MyAnnotation(name="aName", value="aValue") String parameter){
  }
}
```

# Java Reflection - Annotations

- **Parameter Annotations**
  - You can access parameter annotations from the Method object like this:

```
Method method = ... //obtain method object
Annotation[][] parameterAnnotations = method.getParameterAnnotations()
Class[] parameterTypes = method.getParameterTypes();

int i=0;
for(Annotation[] annotations : parameterAnnotations){
  Class parameterType = parameterTypes[i++];

  for(Annotation annotation : annotations){
    if(annotation instanceof MyAnnotation){
        MyAnnotation myAnnotation = (MyAnnotation) annotation;
        System.out.println("param: " + parameterType.getName());
        System.out.println("name : " + myAnnotation.name());
        System.out.println("value: " + myAnnotation.value());
    }
  }
}
```

  - Method.getParameterAnnotations() method returns a two-dimensional Annotation array, containing an array of annotations for each method parameter.

# Java Reflection - Annotations

- **Field Annotations**
  - Example of a field with annotations:

```
public class TheClass {

  @MyAnnotation(name="someName",  value = "Hello World")
  public String myField = null;
}
```

  - You can access field annotations like this:

```
Field field = ... //obtain field object
Annotation[] annotations = field.getDeclaredAnnotations();

for(Annotation annotation : annotations){
    if(annotation instanceof MyAnnotation){
        MyAnnotation myAnnotation = (MyAnnotation) annotation;
        System.out.println("name: " + myAnnotation.name());
        System.out.println("value: " + myAnnotation.value());
    }
}
```

# Java Reflection - Annotations

- **Field Annotations**
  - You can also access a specific field annotation like this:

```
Field field = ... // obtain method object
Annotation annotation = field.getAnnotation(MyAnnotation.class);

if(annotation instanceof MyAnnotation){
    MyAnnotation myAnnotation = (MyAnnotation) annotation;
    System.out.println("name: " + myAnnotation.name());
    System.out.println("value: " + myAnnotation.value());
}
```

# Java Reflection - The Generics Reflection Rule

- Using Java Generics typically falls into one of two different situations:
  - Declaring a class/interface as being parameterizable
  - Using a parameterizable class
- When you write a class or interface you can specify that it should be paramerizable
  - Ex. you can parameterize java.util.List to create a list of say String, like this:

```
List<String> myList = new ArrayList<String>();
```

# Java Reflection - Generic Method Return Types

- Generic Method Return Types
  - If you have obtained a java.lang.reflect.Method object it is possible to obtain information about its generic return type.
  - Here is an example class with a method having a parameterized return type:

```
public class MyClass {

  protected List<String> stringList = ...;

  public List<String> getStringList(){
    return this.stringList;
  }
}
```

  - In this class it is possible to obtain the generic return type of the getStringList() method. In other words, it is possible to detect that getStringList() returns a List<String> and not just a List.

# Java Reflection - The Generics Reflection Rule

- The following piece of code will print out the text "typeArgClass = java.lang.String".
  - The Type[] array typeArguments array will contain one item - a Class instance representing the class java.lang.String. Class implements the Type interface.

```
Method method = MyClass.class.getMethod("getStringList", null);

Type returnType = method.getGenericReturnType();

if(returnType instanceof ParameterizedType){
    ParameterizedType type = (ParameterizedType) returnType;
    Type[] typeArguments = type.getActualTypeArguments();
    for(Type typeArgument : typeArguments){
        Class typeArgClass = (Class) typeArgument;
        System.out.println("typeArgClass = " + typeArgClass);
    }
}
```

# Java Reflection - Generic Method Parameter Types

- You can access the generic parameter types of the method parameters like this:

```
public class MyClass {
  protected List<String> stringList = ...;

  public void setStringList(List<String> list){
    this.stringList = list;
  }
}
```

```
method = Myclass.class.getMethod("setStringList", List.class);

Type[] genericParameterTypes = method.getGenericParameterTypes();

for(Type genericParameterType : genericParameterTypes){
    if(genericParameterType instanceof ParameterizedType){
        ParameterizedType aType = (ParameterizedType) genericParameterType;
        Type[] parameterArgTypes = aType.getActualTypeArguments();
        for(Type parameterArgType : parameterArgTypes){
            Class parameterArgClass = (Class) parameterArgType;
            System.out.println("parameterArgClass = " + parameterArgClass);
        }
    }
}
```

- This code will print out the text "parameterArgType = java.lang.String".
- The Type[] parameterArgTypes array will contain one item - a Class instance representing the class java.lang.String

# Java Reflection - Generic Field Types

- You can access the generic field types like this:

```
public class MyClass {
  public List<String> stringList = ...;
}
```

```
Field field = MyClass.class.getField("stringList");

Type genericFieldType = field.getGenericType();

if(genericFieldType instanceof ParameterizedType){
    ParameterizedType aType = (ParameterizedType) genericFieldType;
    Type[] fieldArgTypes = aType.getActualTypeArguments();
    for(Type fieldArgType : fieldArgTypes){
        Class fieldArgClass = (Class) fieldArgType;
        System.out.println("fieldArgClass = " + fieldArgClass);
    }
}
```

  - This code will print out the text "parameterArgType = java.lang.String".
  - The Type[] array parameterArgTypes array will contain one item - a Class instance representing the class java.lang.String

- https://www.javatpoint.com/DriverManager-class
- http://tutorials.jenkov.com/java-reflection/index.html